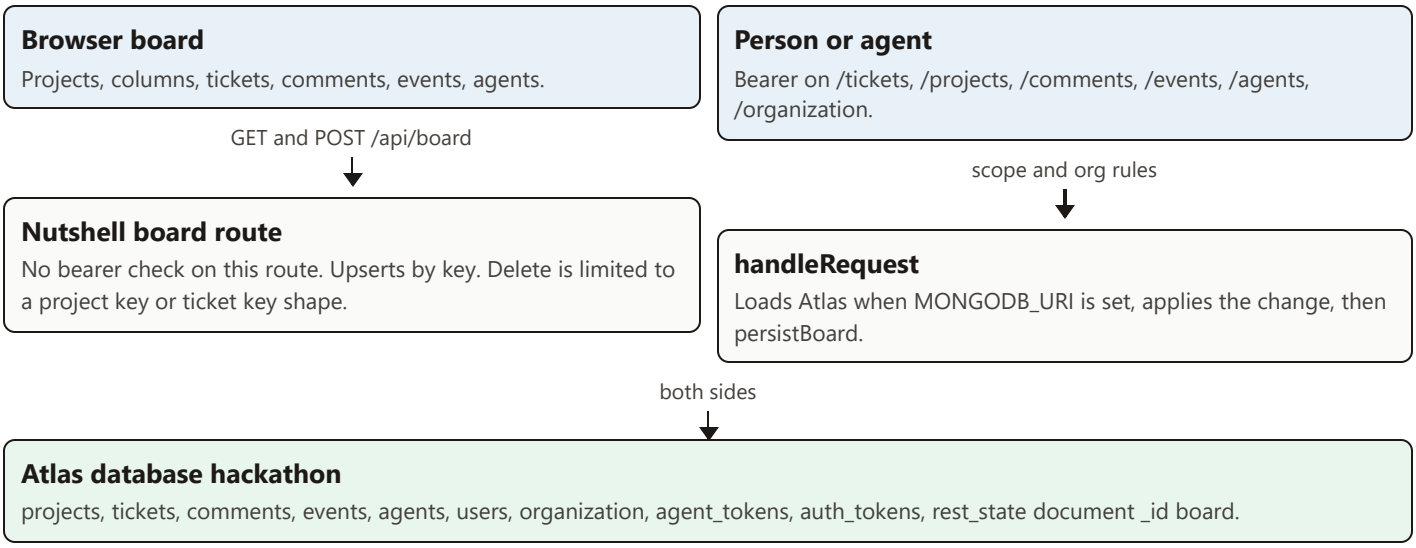
4. Board data flow

Two writers share Atlas. The UI writes the collection documents. The ticket API writes a whole-board snapshot and then mirrors the same collections. Agents and policy go through the ticket API only.



Ticket API save

1. `ensureBoardLoaded` reads `rest_state`, then overlays users, agents, tickets, comments, events, projects, organization settings, and remembered tokens.
2. The store enforces scopes and the three organization rules.
3. `persistBoard` writes `rest_state`, then mirrors projects, tickets, comments, events, users, agents, and organization.
4. Agent bearers are also stored in `agent_tokens` as a SHA-256 `token_hash`. The API keeps the secret in `auth_tokens` so a later request can match it. The document id is the SHA-256 of the secret.
5. A deleted project is noted and then removed from projects, tickets, comments, and events.

Rules on a close

- `approve_before_close` starts on. A new ticket needs approval. Only a human can clear `needs_approval`. A human cannot approve their own ticket. The approver must be a different person from the creator, including when an agent created the ticket for that person.
- `commit_on_move` starts on. An agent status change includes a commit: 7 to 40 hex characters, a message, and non-negative integer line counts. The commit is stored, then the status changes, so one PATCH can satisfy the rule.
- `log_agent_actions` starts on. Agent mutations are appended to events.
- `PATCH /organization` requires a human token. `POST /agents` requires a human token. Issuing or revoking a token requires `tickets:write` and the same owner.

Columns come from the project template. A template is a board, a table, or a roadmap. Status values on the default board are backlog, in_progress, in_review, and done. Ticket keys look like CLW-16.